

AI Risk Manager

Track 02 — Approach Document: Return & Chargeback Risk Scorer

1. Problem Statement

Merchants lose margin to three related but distinct loss types: fraud (stolen identity/cards), product returns, and payment chargebacks. This project targets the **return and chargeback** direction specifically — genuine, authorized transactions that later unwind and cost the merchant money — rather than fraud detection (identity/card theft), which is treated as a separate concern.

The deliverable is a working **risk scorer**: given the details of a transaction at the time it is placed, output a probability that it will end in a return or a chargeback, so the merchant can act on high-risk orders early (extra verification, delayed payout, manual review) instead of absorbing the loss after the fact.

2. Why This Direction

- **Less crowded:** fraud-spike and abuse-ring detection attract most builders; return/chargeback risk is explicitly flagged as under-attracted.
- **Cleaner evaluation:** it is a standard binary classification problem with clear historical labels (returned/charged-back vs. not), making rigorous held-out evaluation straightforward.
- **Clean AI/non-AI separation:** the risk score is pure statistics/ML; an LLM is used only to explain the score in plain language, never to make the decision.
- **Naturally defense-only:** a scoring/flagging system has no offensive capability, satisfying the competition's strict defense-only bar.

3. Scope

In scope	Out of scope
Return-risk scoring	Fraud / identity-theft detection
Chargeback-risk scoring	Abuse-ring / network graph detection
Risk-score explanation (LLM, text only)	Any offense-capable / retaliatory action
Threshold tuning via cost analysis	Automated blocking without human-in-loop review

4. Data & Features

Historical transaction data is used, labeled by actual outcome (returned / charged back / neither). Candidate features:

- **Transaction-level:** order value, payment method, time of purchase, discount/coupon usage.
- **Customer-level:** past return/chargeback rate, account age, billing-shipping address mismatch.
- **Product-level:** category return-rate baseline, price relative to category average, size/fit dependency.
- **Behavioral/velocity:** multiple orders in a short window, repeated failed payment attempts, device/IP reuse across accounts.

5. Methodology

1. **Data preparation:** clean historical orders, label outcome, chronological train/test split so the test set only contains orders later in time than training data.
2. **Feature engineering:** compute the features above per transaction; handle missing/history-less (new customer) cases explicitly.
3. **Model training:** start with logistic regression for interpretability; compare against gradient boosting (XGBoost/LightGBM) for accuracy; select based on held-out performance and cost, not just accuracy.
4. **Threshold selection:** instead of a fixed 0.5 cutoff, sweep thresholds and pick the one that minimizes total business cost (see Section 6).
5. **Explanation layer:** pass the score and top contributing features to an LLM to generate a short, human-readable rationale for the merchant. The LLM never alters the score or decision.

6. Evaluation: Precision, Recall & False-Positive Cost

Evaluation is performed only on the held-out test set, never on training data. Beyond standard precision/recall, each error type is assigned a rupee cost so the model is tuned for real business impact rather than abstract accuracy.

	Actually returned / chargeback	Actually fine
Flagged high-risk	True Positive (TP)	False Positive (FP)
Not flagged	False Negative (FN)	True Negative (TN)

Precision = $TP / (TP + FP)$ — of orders flagged risky, how many actually were.

Recall = $TP / (TP + FN)$ — of all actually risky orders, how many were caught.

Cost of a False Positive: a genuine customer is held/friction'd — estimated as lost-sale probability × average order value, plus a trust/reputation penalty.

Cost of a False Negative: a real return/chargeback is missed — refunded amount + chargeback fee + restocking/logistics cost.

Total Cost = **(FP count × cost per FP) + (FN count × cost per FN)**. The risk threshold is chosen to minimize this total cost, not to maximize accuracy — since the two error types are rarely equally expensive. This cost-based threshold tuning, reported alongside precision/recall, is the key evaluation differentiator for this submission.

7. System Architecture

The pipeline keeps a strict separation: statistics/ML compute and decide the risk score and threshold; the LLM is used only to write a plain-language explanation of a score that has already been computed.

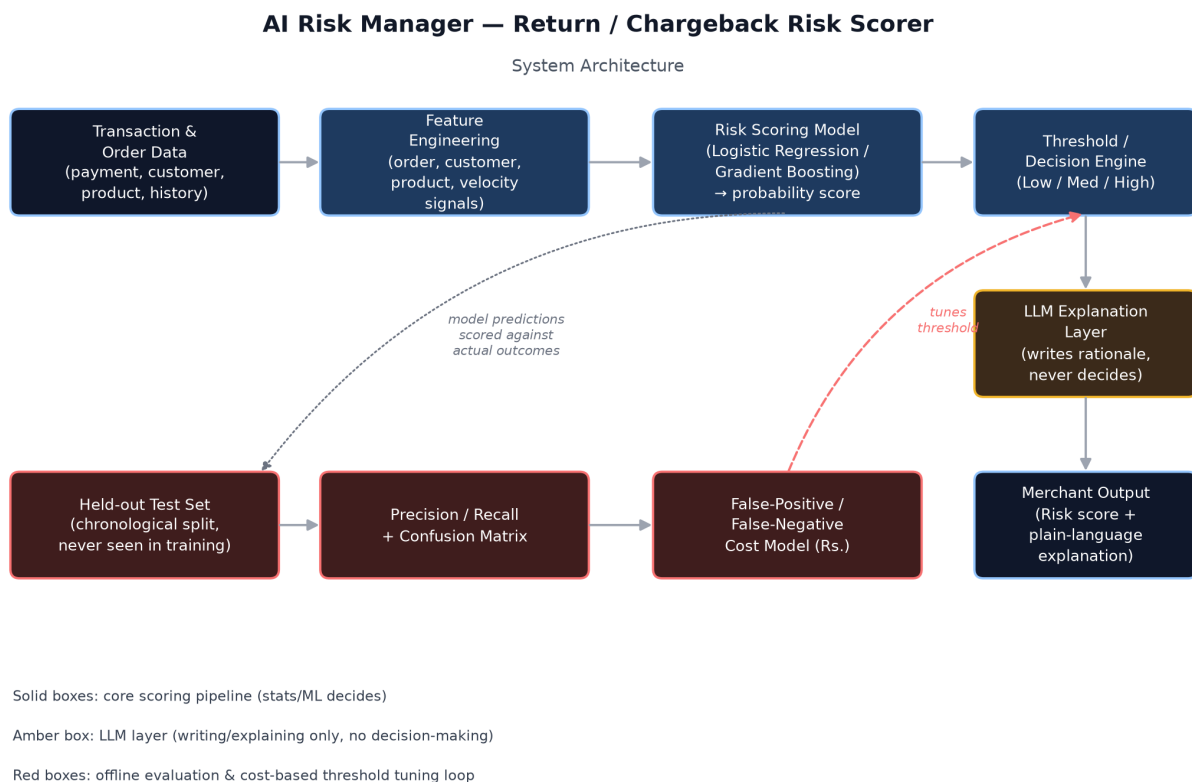


Figure 1: End-to-end architecture — scoring pipeline (top) and offline evaluation / cost-based threshold tuning loop (bottom).

8. Deliverables

- Trained risk-scoring model with documented feature set.
- Held-out test set evaluation report: precision, recall, confusion matrix.
- False-positive / false-negative cost model with a threshold-vs-total-cost chart and the chosen optimal threshold.
- LLM explanation layer producing a short rationale per flagged transaction.
- Short write-up on the AI/non-AI boundary maintained throughout the system.